Phase 7: Integration & External Access

Goal:

Enable Salesforce to securely communicate with external systems (fraud detection API, other services) while ensuring data security, scalability, and compliance with Salesforce governor limits.

---

1. Named Credentials (✔ Used)

Purpose:

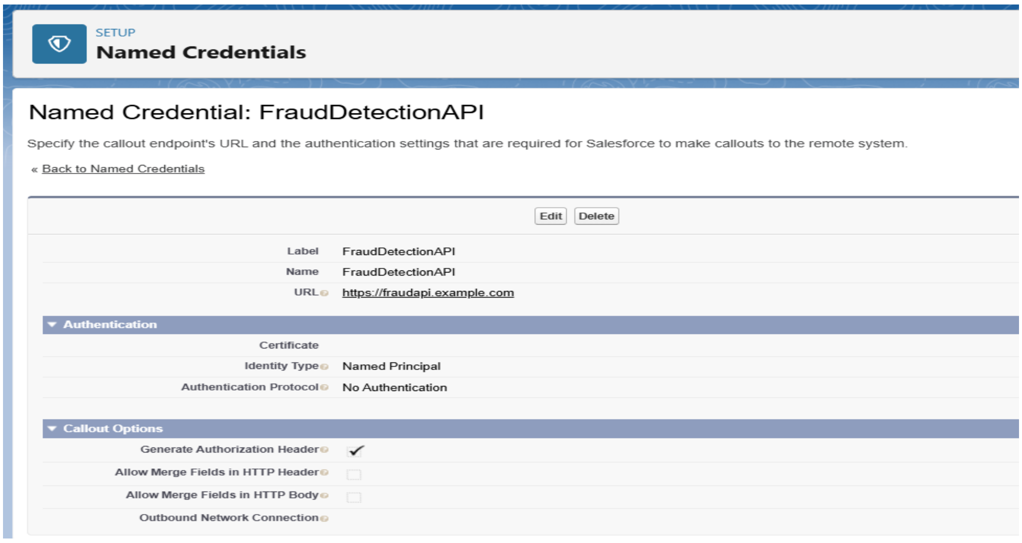
Securely store the base URL and authentication settings for an external API without hardcoding credentials inside Apex classes.

Steps Followed:

1. Setup → Named Credentials → New.
2. Added **Label**: **FraudAPI** and **URL**: **https://fraudapi.example.com.**
3. Chose **Identity Type: Anonymous** and **Authentication Protocol: No Authentication**, since using a demo API without login credentials.
4. Saved the configuration.

Why Important?

- Avoids hardcoding API endpoints in Apex.
- Centralized management of authentication.
- Secure and easy to update if endpoint changes.



2. External Services (❌ Not Used)

Purpose:

Allows Salesforce to consume APIs described by an OpenAPI (Swagger) specification without writing code. Salesforce auto-generates Apex actions from the service schema.

Reason for Skipping:

- Our demo fraud detection API doesn't provide Swagger/OpenAPI definitions.
- Instead, we directly integrated via Named Credentials + Apex Callout.

---

3. Web Services (REST/SOAP) (✔ Used – REST)

Purpose:

Salesforce supports integrating with external systems via REST or SOAP APIs.

- REST → lightweight, JSON-based (used in our project).
- SOAP → XML-based, older format (not used here).

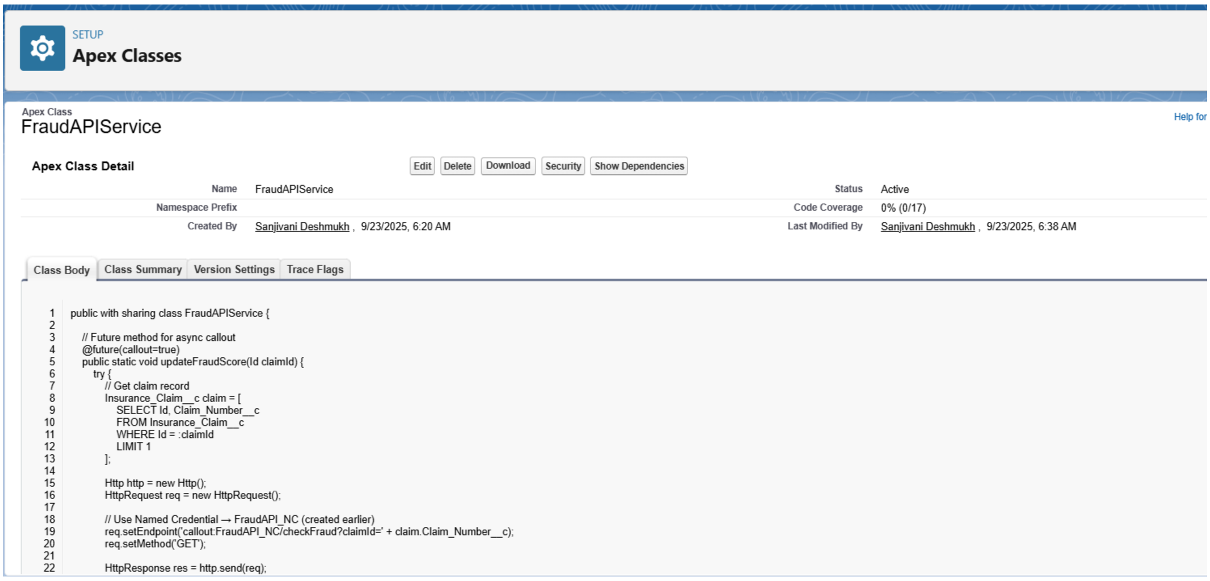
Implementation:

- Created an Apex class with @future(callout=true) to send REST callouts.
- Example: Fetch fraud score for a claim using **req.setEndpoint('callout:FraudAPI/score').**

Code:

```
public with sharing class FraudAPIService {

  // Future method for async callout
  @future(callout=true)
  public static void updateFraudScore(Id claimId) {
    try {
      // Get claim record
      Insurance_Claim__c claim = [
        SELECT Id, Claim_Number__c
        FROM Insurance_Claim__c
        WHERE Id = :claimId
        LIMIT 1
      ];
```

```
            Http http = new Http();
            HttpRequest req = new HttpRequest();

            // Use Named Credential → FraudAPI_NC (created earlier)
            req.setEndpoint('callout:FraudAPI_NC/checkFraud?claimId=' + claim.Claim_Number_
_c);
            req.setMethod('GET');

            HttpResponse res = http.send(req);

            if (res.getStatusCode() == 200) {
                // Parse JSON response
                Map<String, Object> result =
                    (Map<String, Object>) JSON.deserializeUntyped(res.getBody());

                if (result.containsKey('fraudScore')) {
                    Decimal score = Decimal.valueOf(result.get('fraudScore').toString());

                    // Update claim with fraud score
                    claim.Risk_Score__c = score;
                    update claim;
                }
            } else {
                System.debug('Fraud API Callout failed → ' + res.getStatusCode() + ' : ' + res.getBod
y());
            }
        } catch (Exception e) {
            System.debug('Error in FraudAPIService.updateFraudScore: ' + e.getMessage());
        }
    }
}
```



---

4. Callouts (✔ Used)

Purpose:

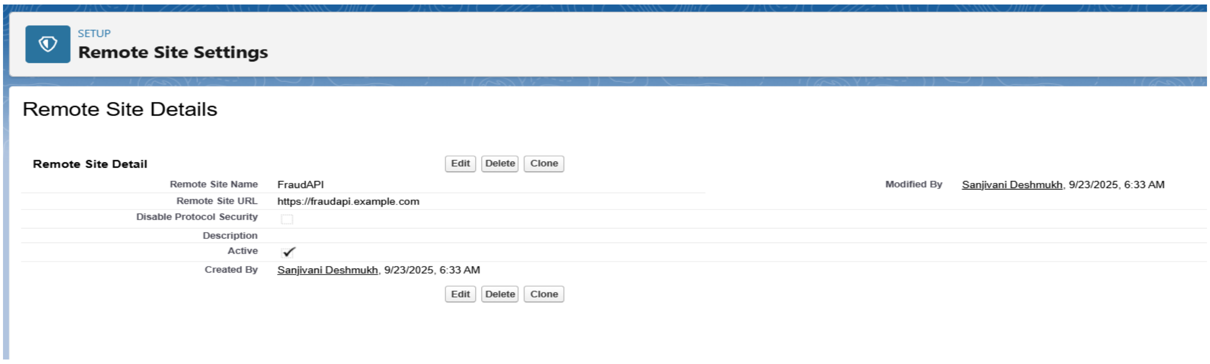
Allow Salesforce to send HTTP requests (GET/POST) to external systems.

Steps:

1. Added Named Credential → FraudAPI.
2. Wrote Apex code:
3. HttpRequest req = new HttpRequest();
4. req.setEndpoint('callout:FraudAPI/score');
5. req.setMethod('GET');
6. HttpResponse res = new Http().send(req);
7. Parsed JSON response and updated Claim Risk Score.

Why Important?

- Enables Salesforce to interact with real-time fraud detection services.
- Provides fraud score enrichment for Insurance Claims.



---

5. Platform Events (✔ Used)

Purpose:

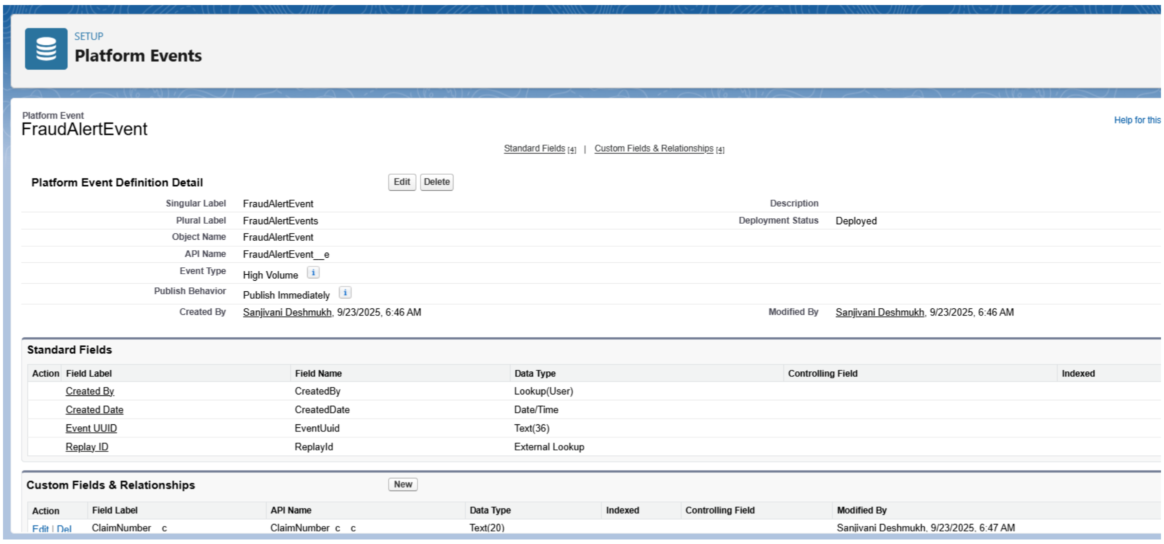
Enable event-driven architecture for real-time communication inside Salesforce and with external systems.

Example in Our Project:

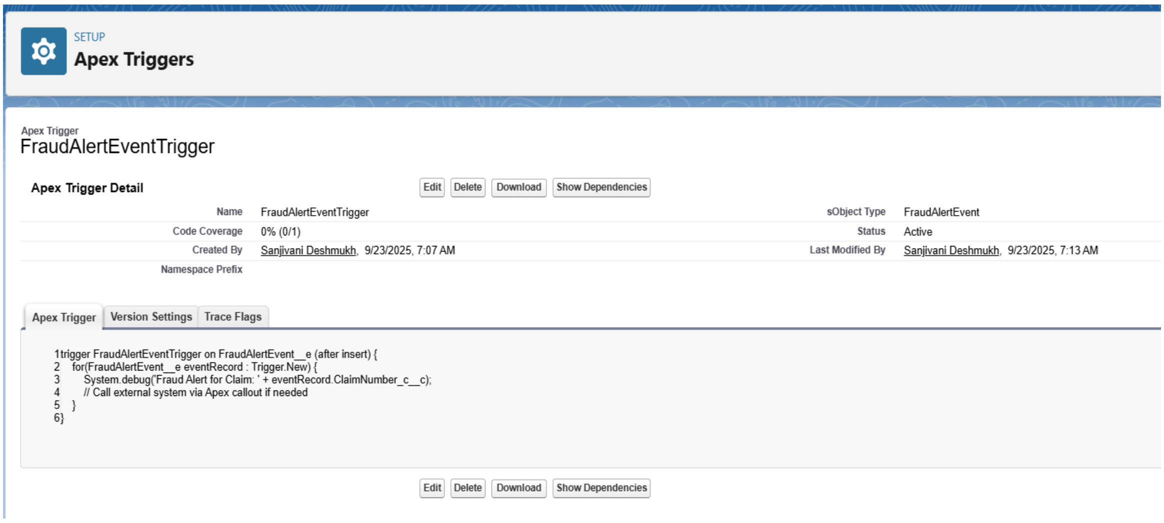
- Created a Fraud_Alert__e platform event.
- Published event when claim fraud score > 80.
- External listeners can react to fraud alerts immediately.

Why Important?

- Decouples fraud detection logic from notifications.
- Supports real-time alerting and system integration.



```
File ▾  Edit ▾  Debug ▾  Test ▾  Workspace ▾  Help ▾  <  >
FraudAlertPublisher.apxc
Code Coverage: None ▾  API Version: 64

public class FraudAlertPublisher {

    // Method to publish the Fraud Alert Event
    public static void publishFraudAlert(String claimNumber, Decimal fraudScore, String customerName) {

        // Create an instance of your Platform Event
        FraudAlertEvent__e alert = new FraudAlertEvent__e(
            ClaimNumber_c__c = claimNumber,
            FraudScore_c__c = fraudScore,
            CustomerName_c__c = customerName,
            Message_c__c = 'High-risk claim detected'
        );

        // Publish the event
        Database.SaveResult result = EventBus.publish(alert);

        if (result.isSuccess()) {
            System.debug('Fraud Alert Event Published Successfully: ' + claimNumber);
        } else {
            System.debug('Failed to publish event: ' + result.getErrors()[0].getMessage());
        }
    }
}
```



---

6. Change Data Capture (❌ Not Used)

Purpose:

Automatically publishes events whenever records in selected objects change.

Reason for Skipping:

- Not essential for our demo project.
- Could be used in future for syncing Claim/Investigation updates with external databases.

---

7. Salesforce Connect (❌ Not Used)

Purpose:

Connects external databases (like Oracle, SAP, AWS) as virtual objects in Salesforce without storing data locally.

Reason for Skipping:

- Our fraud detection service provides API access only, not database connection.
- Salesforce Connect not required.

---

8. API Limits

Purpose:

Salesforce imposes limits to ensure system performance.

Important Limits:

- **REST API Calls per 24 hours** → Based on license (e.g., 15,000 for Enterprise Edition).
- **Callout timeout** → 120 seconds max.
- **Concurrent callouts** → Up to 10 per Apex transaction.

Why Important?

- Helps us design integrations without exceeding limits.
- In real projects, ensures API quota is not exhausted.

---

9. OAuth & Authentication (❌ Not Used – Chose No Auth)

Purpose:
OAuth2 is a standard authentication method for connecting Salesforce to third-party services.

Why Skipped:

- Our fraud detection API is a demo and doesn't require credentials.
- Chose **No Authentication** in Named Credentials.

Note:
In a real system, APIs usually require OAuth2 / Password Authentication for security.

10. Remote Site Settings (✔ Used)

Purpose:

Allow Salesforce to call external services securely.

Steps:

- For Named Credentials → Remote Site Settings are automatically handled.
- If calling endpoint directly, you must manually add base URL in **Setup → Remote Site Settings**.

Why Important?

- Prevents unauthorized external callouts.
- Adds a layer of security by whitelisting allowed domains.